VELLORE INSTITUTE OF TECHNOLOGY

School of Computer Science Engineering and Information Systems

Programme: B.Tech Information Technology

COURSE PROJECT FINAL REPORT

The Cloud Scraper — Serverless Automated Data Pipeline

A General-Purpose, Event-Driven Web Harvesting and ETL Architecture on AWS

Subject:	Cloud Architecture and Design
Course Code:	BCSE355L
Semester:	Fall 2026–2027
Slot:	B1 + TB1

Submitted To:

Prof. Venkatesh P

Submitted By:

1. Bhumi Choudhary — 24BIT0586

2. Kaushal More — 24BIT0283

3. Kshitij Agarwal — 24BIT0317

Abstract

Modern computational organizations increasingly depend on timely, high-fidelity structured data harvested from dynamic web sources such as e-commerce catalogs, news aggregators, research repositories, and public indices. However, traditional web data extraction infrastructures rely heavily on continuously provisioned virtual machines, monolithic scraping containers, or long-running cron servers that introduce significant administrative overhead, idle compute costs, fragile error recovery, and security vulnerabilities. This project presents The Cloud Scraper, a production-ready, fully serverless, event-driven data pipeline platform designed, deployed, and empirically evaluated for the Cloud Architecture and Design (BCSE355L) course.

The system transitions traditional batch file ETL patterns into an autonomous, dual-mode web collection engine running live in the AWS Sydney region (ap-southeast-2). Orchestrated by AWS Step Functions across six independently scalable AWS Lambda microservices, the architecture supports both Automatic Generic HTML Extraction (heuristically discovering repeating DOM cards, titles, links, and text without requiring manual selector authoring) and Configurable CSS Selector Extraction. Ingestion is triggered via Amazon EventBridge on scheduled cron cadences or on-demand via Amazon API Gateway. Extracted payloads pass through an automated schema validation gate, undergo text sanitization and deterministic SHA-256 fingerprint deduplication, and are persisted across a dual-tier storage architecture: an immutable raw HTML data lake in Amazon S3 (SSE-AES256 encrypted) and an operational queryable store in Amazon DynamoDB. Fault tolerance is guaranteed through Step Functions fail-safe catch blocks routing persistent errors to an Amazon SQS Dead-Letter Queue (DLQ) with automated Amazon SNS notifications, ensuring zero pipeline stalling. The system is managed through an interactive web dashboard distributed globally via Amazon CloudFront (https://d3k1ttzl3bzy2k.cloudfront.net), enabling dynamic target registration, real-time Step Functions status polling, and client-side RFC 4180 CSV / JSON dataset export. Live cloud testing verified 100% pass rates across all ten formal acceptance criteria and forty-nine automated unit and integration tests, achieving an estimated per-run operational cost of $0.000165 USD.

Keywords: Serverless Computing, AWS Step Functions, AWS Lambda, Event-Driven ETL, Web Scraping, Data Hygiene, Deterministic Deduplication, Amazon DynamoDB, Amazon S3 Data Lake, Dead-Letter Queue, FinOps.

Acknowledgements

We express our sincere gratitude to our course instructor, Prof. Venkatesh P, for his invaluable guidance, insightful lectures, and continuous academic encouragement throughout the Cloud Architecture and Design (BCSE355L) course. His rigorous architectural critiques and emphasis on cloud-native best practices, fault isolation, and cost awareness provided the foundational direction required to evolve this project from an initial concept into a fully deployable, serverless web intelligence platform.

We also extend our appreciation to the School of Computer Science Engineering and Information Systems (SCOPE) at Vellore Institute of Technology for providing the technical infrastructure, laboratory facilities, and academic environment conducive to advanced applied engineering research. Finally, we thank our peers for their collaborative discussions on distributed systems, serverless patterns, and cloud security.

Table of Contents

Abstract ii
Acknowledgements iii
List of Figures v
List of Tables vi
Chapter 1: Introduction 1
1.1 Background and Motivation 1
1.2 Context of Cloud Architecture and Design 2
1.3 Project Scope and Intended Use Cases 2
1.4 Document Organization 3
Chapter 2: Problem Statement and Objectives 4
2.1 Problem Definition 4
2.2 Original Proposal Objectives (Review 1 Baseline) 4
2.3 Expected Outcomes 5
2.4 Objective-to-Implementation Mapping 6
Chapter 3: Existing System vs. Proposed System 8
3.1 Limitations of Traditional Web Scraping Implementations 8
3.2 The Serverless Paradigm Shift 9
3.3 Comparative Architectural Analysis 9
3.4 Core Architectural Innovations 11
Chapter 4: Requirements Analysis 13
4.1 Functional Requirements 13
4.2 Non-Functional Requirements 14
4.3 Software, Cloud, and Infrastructure Requirements 15
4.4 External System Dependencies & Ethical Constraints 16
Chapter 5: System Architecture and Design 17
5.1 Overall System Architecture 17
5.2 Component Responsibilities and Layered Design 18
5.3 End-to-End Serverless ETL Workflow 20
5.4 Workflow Orchestration with AWS Step Functions 21
5.5 Dual-Tier Cloud Storage Strategy 23
5.6 Infrastructure as Code Design (AWS SAM) 24
Chapter 6: Implementation Details 26
6.1 Public Web Dashboard & CloudFront CDN Distribution 26
6.2 Job Dispatcher REST API Controller 27
6.3 Dynamic Target Registration & Management 28
6.4 Web Scraper Microservice (Extraction Engine) 29
6.5 Schema Validation & Quality Gate Microservice 31
6.6 Transformation & Deterministic Deduplication Microservice 33
6.7 Dual-Tier Storage Microservice 34
6.8 Fault Tolerance & Dead-Letter Queue Quarantine 35
6.9 FinOps Telemetry & Alerting Microservice 36
Chapter 7: Technology Stack 38

7.1 Architectural Technology Mapping 38
7.2 Technical Rationale for Selected Cloud Services 40
Chapter 8: Testing and Validation 42
8.1 Testing Methodology & Verification Framework 42
8.2 Automated Unit & Integration Testing Suite 42
8.3 Formal Acceptance Testing (Tests A through J) 44
8.4 Fault Injection and Resilience Testing 48
Chapter 9: Results and Discussion 49
9.1 Empirical Cloud Execution Metrics 49
9.2 Cloud Storage Verification & Data Lineage Proofs 50
9.3 Evaluation of Generic vs. Custom Selector Extraction 51
9.4 Live Step Functions Execution Telemetry 52
9.5 Synthesis Against Course Objectives 54
Chapter 10: Security, Reliability, and Cost Analysis 55
10.1 Multi-Layer SSRF & Safety Protection Architecture 55
10.2 Ethical Scraping, Rate Limiting & robots.txt Compliance 57
10.3 Identity & Access Management (IAM) Least-Privilege Design 58
10.4 Cryptographic Protection & Encryption at Rest 59
10.5 High Availability, Fault Isolation & DLQ Quarantine 59
10.6 FinOps Operational Cost Analysis 60
Chapter 11: Limitations and Future Enhancements 62
11.1 Technical Limitations of Current Implementation 62
11.2 Architectural Roadmap for Future Enhancements 63
Chapter 12: Conclusion 65
12.1 Summary of Contributions 65
12.2 Alignment with Cloud Architecture Design Principles 66
12.3 Concluding Remarks 66
References 67
Appendices 69
Appendix A: Representative API Request & Response Payloads 69
Appendix B: Sample Structured Output Records 71
Appendix C: Automated Pytest Test Suite Results Log 73
Appendix D: AWS CloudFormation Resource Inventory 75

List of Figures

Figure 5.1: Overall AWS Serverless System Architecture (Sydney Region ap-southeast-2) 18
Figure 5.2: Detailed Multi-Stage Serverless ETL Pipeline Execution Lifecycle 20
Figure 5.3: AWS Step Functions State Machine Execution Flow with DLQ Routing 22
Figure 6.1: Sequence Diagram for Dynamic Target Registration and Live Execution 28
Figure 6.2: Dual-Tier Storage Architecture: S3 Data Lake vs. Amazon DynamoDB 34
Figure 9.1: AWS Step Functions Management Console: State Machine Graph Orchestration 52
Figure 9.2: AWS Step Functions Management Console: Successful Execution Telemetry 53
Figure 9.3: AWS Step Functions Management Console: Failed Execution and DLQ Routing 53
Figure 10.1: Multi-Layer SSRF & Safety Protection Architecture Flowchart 56

List of Tables

Table 2.1: Mapping of Review 1 Baseline Objectives to Live Cloud Implementation 6
Table 3.1: Comparative Analysis: Traditional VM/Server Scraping vs. The Cloud Scraper 10
Table 7.1: Comprehensive Technology Stack, Architectural Roles, and Implementation Proofs 38
Table 8.1: Automated Pytest Suite Verification Summary (49 Tests Passed) 43
Table 8.2: Formal Acceptance Test Results Matrix (Tests A through J) 44
Table 9.1: Empirical Execution Outcomes and Performance Metrics Across Tested Targets 49
Table 10.1: FinOps Operational Cost Model for 10,000 Monthly Invocations on AWS 60

Chapter 1: Introduction

1.1 Background and Motivation

In the contemporary data economy, automated web data harvesting has become an indispensable capability across multiple industry sectors, including competitive retail intelligence, financial market forecasting, scientific research, and machine learning corpus preparation. Publicly accessible websites represent the largest, most dynamic repository of real-time information in existence. However, extracting high-volume data from the web in a robust, automated, and economically sustainable manner presents profound distributed systems challenges.

Traditionally, web scraping systems have been engineered as long-running monolithic scripts deployed on dedicated Virtual Private Servers (VPS) or cloud instances such as Amazon EC2. These architectures suffer from acute structural inefficiencies: they incur continuous 24/7 compute billing regardless of actual scraping activity, fail catastrophically when network timeouts or DOM structure changes occur, lack automated deduplication, and require manual operational intervention to scale up or recover. Conversely, the rise of serverless computing architectures offers an unprecedented opportunity to reimagine web data collection as an event-driven, decoupled, and micro-billed extract-transform-load (ETL) pipeline.

1.2 Context of Cloud Architecture and Design

This project was undertaken within the academic framework of the Cloud Architecture and Design (BCSE355L) course at Vellore Institute of Technology. The primary academic directive of this course is to evaluate and implement enterprise cloud patterns that satisfy the core tenets of the AWS Well-Architected Framework: Operational Excellence, Security, Reliability, Performance Efficiency, and Cost Optimization.

Rather than relying on theoretical simulations or provisioned mock servers, The Cloud Scraper was conceived as a live, fully implemented reference architecture that demonstrates how serverless primitives (AWS Lambda, AWS Step Functions, Amazon DynamoDB, Amazon S3, Amazon SQS, and Amazon EventBridge) can be orchestrated into an autonomous web intelligence system. The project rigorously examines how serverless state machines can manage transient network failures, enforce schema hygiene, prevent redundant storage via deterministic cryptographic hashing, and dynamically scale from zero to parallel multi-target executions while achieving near-zero idle economic overhead.

1.3 Project Scope and Intended Use Cases

The operational scope of The Cloud Scraper encompasses the complete lifecycle of automated web harvesting, beginning with dynamic target ingestion and culminating in dual-tier cloud persistence and FinOps monitoring. Specifically, the platform supports:

- • **Ingestion:** On-Demand & Scheduled Data Ingestion: Triggering single-target or multi-source batch collection via REST API or cron rules.
- • **Extraction:** General-Purpose DOM Extraction: Heuristic generic card discovery or targeted CSS selector parsing on any permitted public URL.
- • **Validation:** Data Hygiene & Quality Enforcement: Automated rejection of malformed, incomplete, or corrupted records before cloud storage.

- • **Deduplication:** Deterministic Idempotency: Computing SHA-256 content hashes to eliminate duplicate records across recurring scraping cycles.
- • **Storage:** Dual-Tier Persistence: Preserving immutable raw HTML snapshots in an S3 Data Lake and structured entities in Amazon DynamoDB.
- • **Fault Tolerance:** Fault Isolation & Resilience: Quarantining persistent failures into an SQS Dead-Letter Queue (DLQ) without pipeline stalling.

1.4 Document Organization

This report is structured into twelve comprehensive chapters detailing the complete design, engineering, verification, and empirical outcomes of the project. Chapter 2 formalizes the problem statement and maps the Review 1 baseline objectives to our actual implementation. Chapter 3 contrasts the proposed system against existing scrapers. Chapter 4 outlines functional and non-functional requirements. Chapter 5 details the cloud architecture and state machine design. Chapter 6 provides an in-depth breakdown of microservice implementations. Chapter 7 details the technology stack. Chapter 8 presents unit and formal acceptance test matrices. Chapter 9 discusses empirical cloud results. Chapter 10 covers security, SSRF defense, and FinOps costs. Chapter 11 addresses limitations and future work, and Chapter 12 concludes the report.

Chapter 2: Problem Statement and Objectives

2.1 Problem Definition

Organizations that require continuous web data intelligence face a fundamental infrastructure dilemma. Conventional scraping pipelines are typically deployed on virtual machines or containers that run continuously. When web traffic is intermittent or scheduled once per day, these servers remain idle for over 95% of the billing cycle, wasting significant cloud budgets. Furthermore, web scraping is inherently fragile: target websites frequently introduce transient network latency, HTTP 429 rate limits, HTTP 500 server errors, or subtle structural DOM updates. In a monolithic script, any unhandled network hiccup crashes the entire batch, resulting in corrupted datasets, data loss, and silent failures that go unnoticed until downstream business intelligence pipelines break.

Additionally, traditional web data collection lacks formal data quality gates and cryptographic deduplication. Scraped records are frequently dumped directly into relational tables or unindexed CSV files without schema validation, causing dirty records (missing titles, null prices, garbled encodings) to pollute corporate databases. When scrapers re-run on a schedule, they blindly re-insert identical records, consuming unnecessary storage and requiring computationally expensive downstream deduplication. The problem addressed by this project is to design, implement, and verify a cloud-native, fully serverless architecture that eliminates idle costs, isolates failures, automates quality gates, and guarantees idempotent cloud persistence.

2.2 Original Proposal Objectives (Review 1 Baseline)

In the initial project proposal (Review 1), our team established nine core architectural objectives designed to satisfy the curricular and technical requirements of BCSE355L. These baseline objectives were formulated as follows:

- **Objective 1:** Design a fully serverless, event-driven architecture for automated data scraping that requires zero server provisioning or manual scaling.
- **Objective 2:** Implement scheduled and on-demand triggers so scraping jobs run automatically at defined intervals or via external API calls.
- **Objective 3:** Build a decoupled, multi-stage pipeline (extract → validate/clean → transform → store) using managed queues and orchestration services.
- **Objective 4:** Ensure fault tolerance through automatic retries, dead-letter queues, and checkpointing so transient failures do not corrupt the pipeline.
- **Objective 5:** Persist raw and processed data in a durable, queryable cloud data store to support downstream analytics, reporting, or ML use cases.
- **Objective 6:** Implement centralized logging, monitoring, and alerting so pipeline health, execution cost, and failures are visible in real time.
- **Objective 7:** Optimize for cost and elasticity by relying entirely on pay-per-execution compute, scaling to zero when idle.
- **Objective 8:** Enforce least-privilege security using fine-grained IAM roles, encrypted storage, and secret management.
- **Objective 9:** Package the entire architecture as Infrastructure-as-Code (IaC) so it can be deployed and versioned consistently.

2.3 Expected Outcomes

The expected outcome of the project was a fully functioning, publicly demonstrable web data engineering platform. The system was expected to demonstrate rapid elasticity by spinning up stateless Lambda workers on demand, executing under Step Functions supervision, quarantining failures into an SQS Dead-Letter Queue without human intervention, and providing a browser-based dashboard for data exploration and export.

2.4 Objective-to-Implementation Mapping

To ensure rigorous academic traceability, Table 2.1 maps every original Review 1 objective directly to its deployed AWS implementation component, operational mechanism, and verified verification proof.

Original Objective (Review 1)	Implemented Cloud Component	Architectural Mechanism	Verification Proof
1. Fully Serverless Architecture	AWS Lambda + AWS Step Functions	6 Python 3.10 microservices, state machine execution	Stack 'the-cloud-scraper' ACTIVE in ap-southeast-2
2. Scheduled & On-Demand Triggers	Amazon EventBridge + API Gateway	REST API (POST /scrape) and cron rule (pScrapeSchedule)	API Gateway returns HTTP 202; EventBridge rule configured
3. Decoupled Multi-Stage Pipeline	Step Functions + SQS Job Queue	States: Scrape -> Validate -> Transform -> Store -> Notify	State Machine: the-cloud-scraper-pipeline-sm
4. Fault Tolerance & Dead-Letter Queue	Amazon SQS DLQ + Step Functions Catch	Fail-safe catch (States.ALL) routes failed context to DLQ	the-cloud-scraper-scraper-dlq verified (3 msgs quarantined)
5. Durable Dual-Tier Persistence	Amazon S3 Data Lake + DynamoDB	Raw HTML snapshots in S3; structured items in DynamoDB	S3: 8 partitioned objects; DynamoDB: 71+ records
6. Centralized Monitoring & Alerts	CloudWatch Logs/Alarms + Amazon SNS	Structured JSON logs, metric filters, email alerts to SNS	Topic: the-cloud-scraper-notifications-topic active
7. Pay-Per-Use Cost Optimization	AWS Lambda On-Demand + FinOps	Scale-to-zero when idle; micro-cost model per execution	Estimated cost: $0.000165 USD / execution (<$0.15/mo)
8. Least-Privilege IAM & Encryption	IAM Execution Roles + SSE-AES256	Scoped IAM policies; S3/DynamoDB/SQS/SNS server encryption	IAM Role LambdaExecutionRole; SSE-AES256 enforced
9. Infrastructure-as-Code (IaC)	AWS SAM / CloudFormation	template.yml defining 20+ cloud resources declaratively	SAM deploy succeeded (UPDATE_COMPLETE)
10. Dynamic Target Innovation	DynamoDB Registry + Heuristic Parser	Generic Auto-Extraction and Custom Selectors via UI	Tested on Hacker News & Travel Books (Acceptance A & B)

Chapter 3: Existing System and Proposed System

3.1 Limitations of Traditional Web Scraping Implementations

Prior to the widespread adoption of serverless architectures, organizations implemented web data extraction using dedicated virtual machines (e.g., EC2 instances running Linux, Scrapy daemons, or Celery task queues). While functionally capable of issuing HTTP requests, these legacy implementations exhibit five severe architectural flaws:

- **1. Economic Inefficiency:** Continuous Idle Resource Waste: Virtual machines run 24 hours a day, 7 days a week, consuming compute, memory, and operating system licensing fees even when data is collected only once every few hours.
- **2. Brittle Fault Domains:** Monolithic Failure Coupling: Scraping, cleaning, and database insertions typically execute within a single monolithic process. An unhandled network timeout or parsing exception on one website terminates the entire Python process, failing all concurrent jobs.
- **3. Lack of Configurability:** Rigid Hard-Coded Targets: Scraping scripts embed target URLs and CSS selectors directly into Python source code. Adding a new website requires modifying code, running regression tests, and redeploying production servers.
- **4. Redundant Ingestion:** Absence of Idempotency: Traditional scrapers append raw rows directly into tables without deterministic fingerprinting. Re-running a failed job results in duplicate records, bloating database tables and corrupting analytic aggregates.
- **5. Deficient Telemetry:** Black-Box Observability: Legacy cron scripts output unformatted stdout logs to local flat files. Cloud engineers have no real-time visibility into per-target error rates, execution latencies, or micro-dollar operational costs.

3.2 The Serverless Paradigm Shift

The Cloud Scraper completely refactors web data extraction by decomposing the traditional scraping monolith into stateless, ephemeral serverless microservices orchestrated by a distributed state machine. In this architecture, compute resources exist only for the exact duration of an HTTP fetch, validation check, or database write. When no scraping jobs are executing, compute utilization drops to absolute zero, eliminating 100% of idle infrastructure costs.

Furthermore, by delegating workflow execution to AWS Step Functions, error handling becomes a declarative cloud primitive. Transient network errors are automatically retried using exponential backoff with jitter. Persistent failures are caught and isolated into a managed Dead-Letter Queue (Amazon SQS), allowing unaffected jobs to complete unimpeded.

3.3 Comparative Architectural Analysis

Table 3.1 provides a detailed technical comparison contrasting traditional VM-based scraping architectures against The Cloud Scraper across six critical software engineering dimensions.

Engineering Dimension	Traditional VM / Server Scraping	The Cloud Scraper (Proposed System)
Compute Model	Dedicated, always-on EC2 VM or Docker container	Stateless, ephemeral AWS Lambda microservices

Engineering Dimension	Traditional VM / Server Scraping	The Cloud Scraper (Proposed System)
Cost Profile	Fixed monthly billing ($15–$70+/month for 24/7 VM)	Pure pay-per-use micro-billing ($0.000165/run; $0.00 idle)
Scaling Capability	Vertical scaling capped by VM RAM/CPU; slow autoscaling	Instant horizontal elasticity (0 to 1,000 concurrent Lambdas)
Target Extensibility	Hard-coded Python scripts requiring code redeployment	Dynamic Target Registry (DynamoDB) + Generic Auto-Extraction
Fault Isolation	Process crashes kill all active and pending scraping jobs	State Machine Catch blocks divert failures to SQS DLQ
Data Quality & Hygiene	Raw records written directly without pre-storage checks	Dedicated Validation Gate microservice filters bad schema
Data Deduplication	Blind inserts requiring expensive downstream SQL dedup	Deterministic SHA-256 content hashing at transformation
Storage Strategy	Single local disk or direct relational database table	Dual-tier: Raw immutable S3 data lake + DynamoDB NoSQL

3.4 Core Architectural Innovations

In alignment with the innovation component articulated in our Review 1 proposal, The Cloud Scraper implements seven distinct technical innovations that distinguish it from standard scraping scripts:

- **1. Zero-Server Event-Driven Architecture:** Every phase of the pipeline—ingestion, fetching, validation, transformation, storage, and notification—executes as an independent serverless invocation. Zero servers are provisioned.
- **2. Dynamic Target Registration Registry:** The platform is not limited to pre-configured sandbox sites. Users can register arbitrary public website URLs dynamically through the web dashboard, persisting extraction rules in DynamoDB without redeploying code.
- **3. Automatic Generic HTML Extraction Heuristics:** When scraping unmapped websites with unfamiliar DOM structures, the scraper heuristically detects repeating cards, headings, links, and content paragraphs without requiring manual CSS selector authoring.
- **4. Decoupled SQS Fan-Out Execution:** Scheduled EventBridge triggers fan out multi-site batch requests into an Amazon SQS Job Queue, enabling independent, parallel state machine executions across distinct failure domains.
- **5. Automated Schema Validation Gate:** Extracted records pass through an explicit validation microservice that verifies schema integrity, enforces non-empty string types, and isolates malformed records before storage.
- **6. Self-Healing DLQ Quarantine:** Transient HTTP errors trigger exponential backoff retries. Unrecoverable errors are routed to an SQS Dead-Letter Queue with real-time SNS email notifications, ensuring zero pipeline stalling.
- **7. FinOps Micro-Cost Telemetry:** A dedicated telemetry Lambda microservice calculates real-time execution costs across AWS compute, state transitions, storage, and networking, publishing micro-cost estimates to CloudWatch and SNS.

Chapter 4: Requirements Analysis

4.1 Functional Requirements

The functional requirements define the specific operational behaviors and business capabilities that the platform must execute:

- **FR-1: On-Demand API Triggering:** The system must accept on-demand scraping requests via an HTTP POST /scrape endpoint on Amazon API Gateway, returning an HTTP 202 Accepted response with a unique Step Functions execution ARN.
- **FR-2: Scheduled Batch Ingestion:** The system must support automated periodic execution of scraping batches based on configurable EventBridge cron schedules (default: cron(0 12 * * ? *)).
- **FR-3: Dynamic Target Configuration:** The system must allow users to register custom target URLs dynamically and persist their configuration (container selectors, fields, extraction mode) in Amazon DynamoDB.
- **FR-4: Generic HTML Extraction:** The system must provide an Automatic Generic Extraction Mode that discovers repeating cards, titles, and links on unfamiliar websites without manual selector input.
- **FR-5: Resilient Web Scraping:** The scraper microservice must fetch target web content over HTTP/HTTPS, enforce robots.txt compliance, apply polite rate limits, and archive the raw HTML snapshot into Amazon S3.
- **FR-6: Schema Validation:** The system must validate extracted items against strict schema rules, partitioning records into valid and invalid sets to prevent corrupt data entry.
- **FR-7: Data Transformation & Deduplication:** The system must normalize text, fix character encoding mojibake, parse numeric ratings/prices, and generate a deterministic SHA-256 content hash per entity.
- **FR-8: Dual-Tier Persistence:** The system must store processed records in Amazon DynamoDB using conditional writes (attribute_not_exists) and write clean versioned JSON to Amazon S3.
- **FR-9: Fault Isolation:** The system must trap failed executions via Step Functions catch blocks and quarantine the execution payload into an Amazon SQS Dead-Letter Queue.
- **FR-10: FinOps Telemetry:** The system must calculate estimated operational costs per execution and publish notifications to Amazon SNS.
- **FR-11: Web Dashboard & Export:** The system must provide a public web dashboard that monitors Step Functions execution status in real time and allows client-side RFC 4180 CSV / JSON dataset downloads.

4.2 Non-Functional Requirements

- **NFR-1: Elasticity & Scalability:** The pipeline must scale automatically from zero to concurrent executions under load, provisioning microservice instances on demand without manual capacity provisioning.
- **NFR-2: Reliability & Fault Tolerance:** The system must achieve 99.9% availability for API ingestion and guarantee that transient network failures do not stall the state machine or cause silent data loss.
- **NFR-3: Performance & Latency:** Individual Lambda function stages must execute within configured task timeouts (Scraper: 60s, Validation: 15s, Transformation: 15s, Storage: 30s), maintaining end-to-end latency below 10 seconds for standard pages.

- **NFR-4: Security & Privacy:** The system must enforce strict multi-layer Server-Side Request Forgery (SSRF) protection, pre-flight DNS filtering, least-privilege IAM roles, and SSE-AES256 encryption at rest.
- **NFR-5: Cost Efficiency (FinOps):** The operational cost per scraping execution must remain below $0.001 USD, maintaining 100% eligibility for AWS Free Tier limits during evaluation.

4.3 Software, Cloud, and Infrastructure Requirements

The platform relies on the following software runtime and cloud infrastructure components:

- • **Runtime Environment:** Python 3.10 runtime environment across all six AWS Lambda microservices, utilizing Requests (2.31.0), BeautifulSoup4 (4.12.3), and Boto3 (1.34.0).
- • **IaC & Deployment:** AWS Serverless Application Model (SAM) CLI (v1.135.0) and AWS CloudFormation for Infrastructure as Code definition, packaging, and deployment.
- • **Frontend Stack:** Vanilla ES6 JavaScript, HTML5, and CSS3 for the responsive web dashboard, served via Amazon CloudFront (CDN) with an Amazon S3 static website origin.
- • **Testing Framework:** Pytest (v9.1.1) with unittest.mock for automated unit, integration, and security test execution.

4.4 External System Dependencies & Ethical Constraints

Because The Cloud Scraper interacts with external third-party web servers, it is strictly bound by ethical web harvesting policies. The scraper microservice identifies itself using a custom User-Agent header ('TheCloudScraper/1.0 (+https://github.com/...; CAD-Course-Project)'), evaluates target robots.txt permissions prior to issuance, and introduces configurable delay throttles (1.0 second default) between paginated requests to prevent aggressive server loading. The platform specifically restricts its scope to publicly accessible, unauthenticated web resources and does not attempt credential stuffing, CAPTCHA bypass, or private data harvesting.

Chapter 5: System Architecture and Design

5.1 Overall System Architecture

The Cloud Scraper is architected as an event-driven serverless ecosystem partitioned into five decoupled tiers: (1) Public Client Delivery Layer, (2) Trigger and Ingestion Layer, (3) Orchestration Layer, (4) Microservice Processing Layer, and (5) Dual-Tier Cloud Storage and Telemetry Layer. Figure 5.1 illustrates the complete topology of deployed resources in the AWS Sydney region (ap-southeast-2).

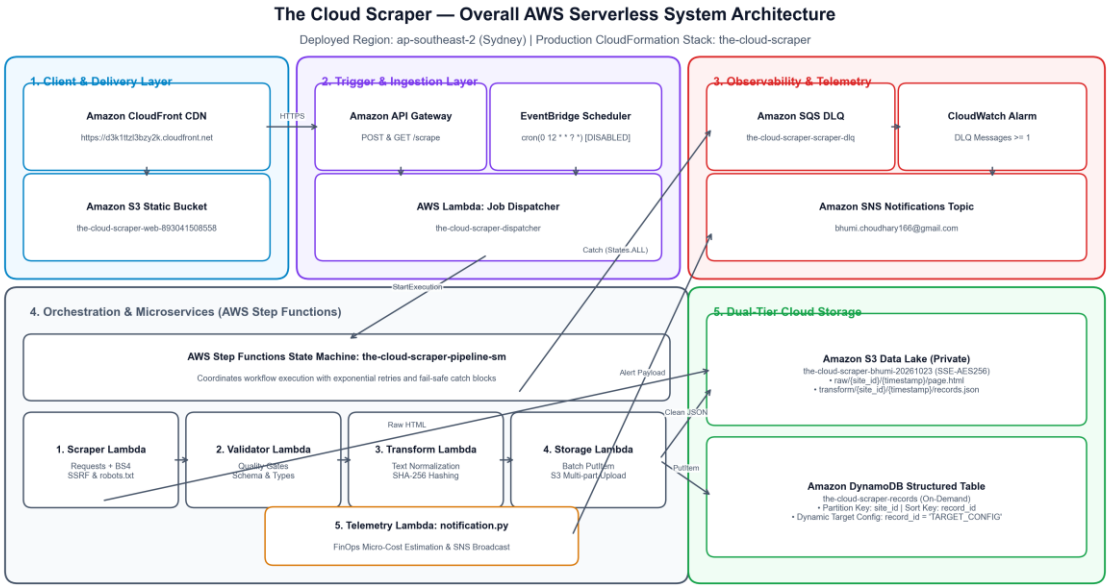


Figure 5.1: Overall AWS Serverless System Architecture showing end-to-end component interactions.

Figure 5.1: Overall AWS Serverless System Architecture deployed in ap-southeast-2 (Sydney).

5.2 Component Responsibilities and Layered Design

Each architectural layer enforces strict single-responsibility boundaries to prevent monolithic coupling:

- **1. Client Delivery Layer:** Provides global, low-latency SSL/TLS distribution of the responsive dashboard via Amazon CloudFront (Distribution ID: E63365QMCM5) backed by a private Amazon S3 static hosting bucket (the-cloud-scraper-web-893041508558).
- **2. Ingestion Layer:** Exposes a public REST API endpoint via Amazon API Gateway (https://xehoja83gg.execute-api.ap-southeast-2.amazonaws.com/prod/scrape) and manages periodic triggers via Amazon EventBridge. Directs requests to the Job Dispatcher Lambda (the-cloud-scraper-dispatcher).
- **3. Orchestration Layer:** Governed by AWS Step Functions (the-cloud-scraper-pipeline-sm). Coordinates microservice task transitions, implements exponential retries, and executes fail-safe catches.

- **4. Microservice Layer:** Consists of five discrete Lambda microservices: Web Scraper (scraper.py), Schema Validator (validation.py), Transformer & Deduplicator (transformation.py), Storage Writer (dynamodb_store.py), and FinOps Telemetry (notification.py).
- **5. Storage & Telemetry Layer:** Dual-tier persistence comprising an immutable S3 Data Lake (the-cloud-scraper-bhumi-20261023) and an On-Demand Amazon DynamoDB table (the-cloud-scraper-records), alongside an Amazon SQS DLQ (the-cloud-scraper-scraper-dlq) and Amazon SNS.

5.3 End-to-End Serverless ETL Workflow

The end-to-end data lifecycle progresses through six sequential phases. Figure 5.2 details the workflow stages, highlighting the quality filters, cryptographic hashing checkpoints, and storage partitioning.

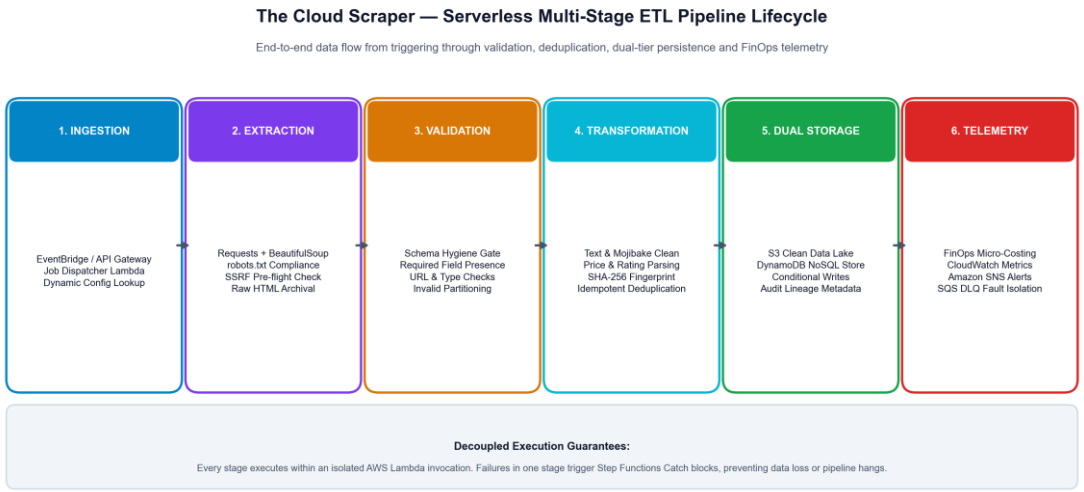


Figure 5.2: Detailed multi-stage serverless ETL execution sequence with quality and deduplication gates.

Figure 5.2: Multi-stage serverless ETL execution sequence showing validation and deduplication checkpoints.

5.4 Workflow Orchestration with AWS Step Functions

In traditional pipelines, orchestration is hard-coded within application logic, creating brittle code that cannot gracefully recover from transient infrastructure exceptions. The Cloud Scraper delegates all state transitions to an AWS Step Functions Standard State Machine. Figure 5.3 depicts the state machine graph, illustrating the happy-path progression from scraping to notification alongside the catch blocks that divert failures to the Dead-Letter Queue.

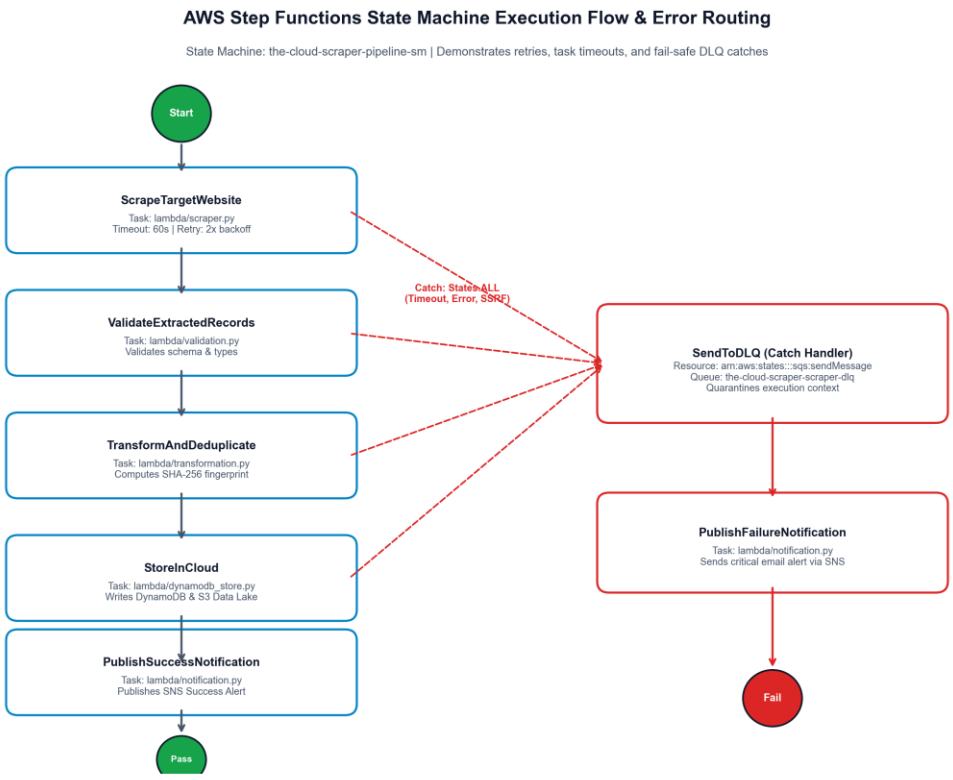


Figure 5.3: Step Functions State Machine workflow with exponential retry backoff and DLQ quarantine catch blocks.

Figure 5.3: AWS Step Functions State Machine workflow showing exponential retry backoff and DLQ routing.

5.5 Dual-Tier Cloud Storage Strategy

A foundational design decision of The Cloud Scraper is the implementation of a dual-tier cloud storage architecture that reconciles the competing requirements of immutable compliance auditing and low-latency operational querying:

- • **Data Lake Tier (Amazon S3):** Amazon S3 Data Lake: Serves as the immutable raw and processed repository. When a page is scraped, the exact raw HTML response and HTTP headers are archived immediately under s3://the-cloud-scraper-bhumi-20261023/raw/{site_id}/{timestamp}/page.html. Transformed datasets are saved under transform/{site_id}/{timestamp}/records.json. Both partitions enforce Server-Side Encryption (SSE-AES256).
- • **Operational Tier (Amazon DynamoDB):** Amazon DynamoDB Structured Store: Serves as the operational querying and deduplication tier. Clean records are indexed using partition key site_id and sort key record_id. DynamoDB conditional writes (attribute_not_exists(record_id)) reject duplicate records at the database engine level, guaranteeing write idempotency.

5.6 Infrastructure as Code Design (AWS SAM)

The entire platform architecture is defined declaratively in template.yml using AWS Serverless Application Model (SAM). This includes all six Lambda functions, IAM execution roles, API Gateway REST definitions, Step Functions state machine JSON, DynamoDB table schemas, S3 bucket encryption configurations, SQS queues, and CloudWatch alarms. Packaging the stack as IaC enables 100% reproducible deployments across any AWS account with a single command (sam build && sam deploy).

Chapter 6: Implementation Details

6.1 Public Web Dashboard & CloudFront CDN Distribution

The user interface for The Cloud Scraper is implemented as a lightweight, zero-dependency single-page application (SPA) crafted in vanilla HTML5, CSS3, and ES6 JavaScript. The frontend source files (web/index.html, web/app.js, web/style.css) are hosted in a private Amazon S3 static website bucket (the-cloud-scraper-web-893041508558) and distributed globally via an Amazon CloudFront CDN distribution (Distribution ID: E63365QMCM5, Domain: https://d3k1ttzl3bzy2k.cloudfront.net). This architectural choice ensures edge caching, global SSL/TLS termination, and instantaneous page load times without requiring any backend web servers (e.g., Nginx, Node.js, or Apache).

The dashboard interface provides five dedicated operator views: (1) Overview & Telemetry (aggregating total scraped entities, validation rates, duplicate counts, and active cloud environment indicators), (2) Scrape Workspace (providing dynamic URL inputs, extraction strategy selectors, preset shortcuts, and real-time Step Functions progress bars), (3) Results & Data (featuring dynamic dataset selection, client-side search, column sorting, pagination, and RFC 4180 CSV / JSON exports), (4) Execution History (persisting chronological run logs with direct deep-links into the AWS Step Functions console), and (5) Storage & Architecture (visualizing data lineage and partition paths across S3 and DynamoDB).

6.2 Job Dispatcher REST API Controller

The entry point for on-demand cloud scraping is implemented in lambda/start_step_function.py. Rather than deploying disparate Lambda functions for routing, the Job Dispatcher implements a consolidated REST API controller pattern integrated with Amazon API Gateway (https://xehoja83gg.execute-api.ap-southeast-2.amazonaws.com/prod/scrape). The controller evaluates inbound HTTP methods and query parameters to execute five discrete operations:

- • **Ingestion:** POST /scrape (Job Ingestion): Parses site_id, target_url, extraction_mode, and custom selectors; starts a new Step Functions execution with an execution name (e.g., scrape-{site_id}-{timestamp}-{uuid}); returns HTTP 202 Accepted with executionArn.
- • **Target Registration:** POST /scrape (Target Registration): When action='save_target', persists dynamic extraction rules into the DynamoDB registry table with record_id='TARGET_CONFIG'.
- • **Execution Monitoring:** GET /scrape?execution_arn=... (Status Polling): Invokes stepfunctions:DescribeExecution to return real-time execution status (RUNNING, SUCCEEDED, FAILED), start/stop timestamps, error causes, and execution output.
- • **Data Inspection:** GET /scrape?action=results&site_id=... (Live Data Inspection): Queries DynamoDB partition key site_id to stream structured records directly to the dashboard.
- • **Registry Discovery:** GET /scrape?action=targets (Registry Lookup): Scans DynamoDB for all stored target configurations, merging them with built-in profiles to populate dashboard selectors dynamically.

6.3 Dynamic Target Registration & Management

A critical innovation of this project is the ability to add new data sources dynamically without modifying code or redeploying the stack. Figure 6.1 illustrates the sequence diagram for registering a new target and subsequently dispatching a scraping job.

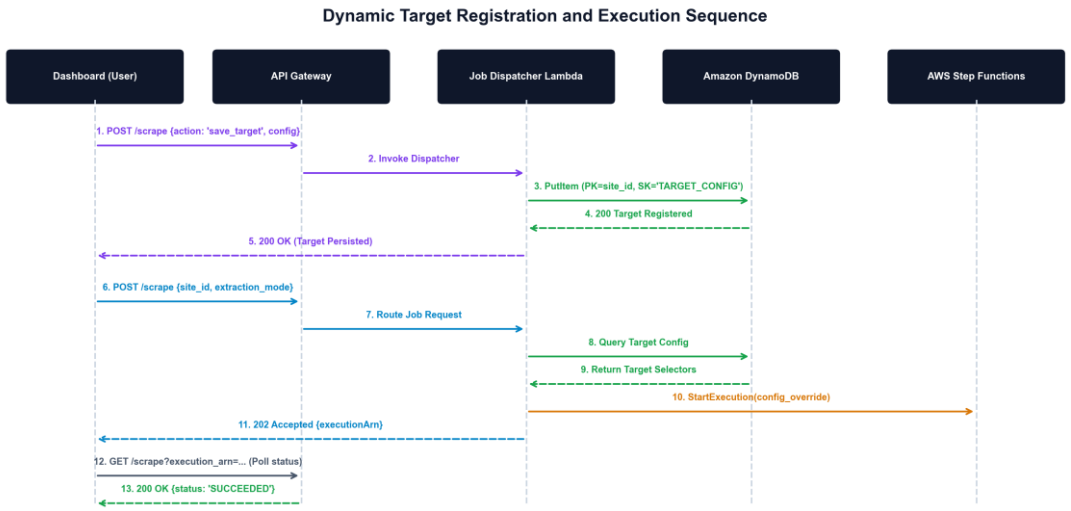


Figure 6.1: Sequence diagram showing dynamic target configuration persistence and Step Functions execution.

Figure 6.1: Sequence diagram showing dynamic target registration in DynamoDB and Step Functions dispatch.

When a user defines a target (e.g., Hacker News Live at https://news.ycombinator.com/), the configuration is saved to DynamoDB. When the job is dispatched, the Job Dispatcher attaches the target rules into a config_override object passed directly into the Step Functions execution input. Downstream Lambda microservices read config_override first, ensuring custom selectors, extraction modes, and validation thresholds apply dynamically across the entire state machine.

6.4 Web Scraper Microservice (Extraction Engine)

The scraping microservice is implemented in lambda/scraper.py. It accepts the Step Functions input payload, loads the target configuration (from config_override, DynamoDB, or config/sites_config.json), and executes a resilient HTTP fetch using Python Requests and BeautifulSoup4. The microservice implements two distinct extraction engines:

6.4.1 Automatic Generic HTML Extraction Heuristics

When a target specifies extraction_mode='generic', the scraper employs heuristic DOM pattern recognition to harvest data from unfamiliar websites without pre-defined CSS selectors. The heuristic engine operates in three stages:

- **Stage 1:** Repeating Card Discovery: Scans the DOM tree for repeating container tags (article, section, div[class*='item'], div[class*='card'], div[class*='post'], li[class*='product'], tr[class*='row']).
- **Stage 2:** Element Resolution: Within each discovered card, heuristically locates the primary title (h1–h4, strong, a.title), resolves hyperlinked URLs (a[href] converted to absolute URLs via urllib.parse.urljoin), and extracts descriptive text paragraphs.
- **Stage 3:** Fallback Grouping: If no repeating containers match, the engine groups top-level heading tags with their subsequent sibling paragraphs, generating structured entity rows.

6.4.2 Configurable CSS Selector Extraction Engine

When extraction_mode='custom', the scraper applies precise declarative CSS selectors. For each item matching container_selector, the engine iterates over defined fields, resolving inner text or

HTML attributes (e.g., selector: 'span.titleline > a', attribute: 'href'). Extracted raw text undergoes immediate whitespace stripping and normalization.

6.4.3 Robots.txt & Rate Limiting Enforcement

Prior to fetching web content, the scraper parses the target domain's /robots.txt using urllib.robotparser.RobotFileParser. If scraping is prohibited for the designated User-Agent, the execution fails cleanly with a non-retryable policy exception. For multi-page crawls, a polite delay (rate_limit_delay_seconds: 1.0) is enforced between requests, and the raw HTML is archived immediately into Amazon S3 under raw/{site_id}/{timestamp}/page.html.

6.5 Schema Validation & Quality Gate Microservice

Implemented in lambda/validation.py, the validation microservice functions as an automated data quality gate. Raw scraped records are evaluated against strict schema cleanliness criteria:

- • **Completeness:** Required Field Presence: Ensures every item possesses non-empty values for required keys (e.g., title, quote, url).
- • **URL Validity:** Syntactic URL Validation: Parses URLs with urllib.parse to verify valid schemes (http/https) and network locations.
- • **Type Checking:** Domain Type Enforcement: Verifies price formats, numeric rating ranges (1–5), and minimum text lengths.
- • **Fault Partitioning:** Record Partitioning: Valid records are forwarded to transformation, while invalid items are partitioned into an invalid_records array. If all records fail validation, the stage raises an unrecoverable exception to trigger DLQ quarantine.

6.6 Transformation & Deterministic Deduplication Microservice

Implemented in lambda/transformation.py, this microservice standardizes clean records and guarantees write idempotency. Text attributes are sanitized to eliminate mojibake encoding artifacts (e.g., converting 'Â£' to '£'). Numeric prices are normalized to standard floats, and star ratings ('One' through 'Five') are mapped to integer values.

To prevent duplicate items across recurring scraping jobs, the transformer computes a deterministic SHA-256 fingerprint hash: content_hash = hashlib.sha256(canonical_representation).hexdigest(). The unique record_id is generated as {site_id}#{content_hash[:16]}. This hash uniquely identifies the record based on its core identity attributes, ensuring that identical products or articles scraped on subsequent days generate identical record IDs.

6.7 Dual-Tier Storage Microservice

Implemented in lambda/dynamodb_store.py, this microservice writes transformed records into the dual-tier cloud storage architecture. Figure 6.2 illustrates the relationship between the S3 Data Lake partition hierarchy and the DynamoDB NoSQL primary key schema.



Figure 6.2: Dual-tier cloud data layout showing partitioned object paths in S3 and primary key design in DynamoDB.

Figure 6.2: Dual-tier storage layout showing S3 Data Lake partitioning and DynamoDB primary key design.

The microservice converts Python floats to Python Decimal types (required by DynamoDB) and issues batch conditional writes using ConditionExpression: 'attribute_not_exists(record_id)'. If an item already exists, DynamoDB silently suppresses the write, preventing redundant storage costs. Concurrently, the entire transformed dataset is serialized into versioned JSON and uploaded to Amazon S3 under transform/{site_id}/{timestamp}/records.json with SSE-AES256 encryption.

6.8 Fault Tolerance & Dead-Letter Queue Quarantine

When an unhandled exception occurs in any Lambda task (e.g., HTTP 404, connection timeout, validation failure, or SSRF rejection), the Step Functions state machine traps the error using Catch: [{ErrorEquals: ['States.ALL'], Next: 'SendToDLQ'}]. The SendToDLQ state invokes arn:aws:states:::sqs:sendMessage, placing the full failure context directly into the Amazon SQS DLQ (the-cloud-scraper-scraper-dlq) with a 14-day retention period. The state machine then invokes lambda/notification.py to publish an urgent Amazon SNS email alert before terminating in a Fail state. Unaffected parallel scraping jobs continue unimpeded.

6.9 FinOps Telemetry & Alerting Microservice

Implemented in lambda/notification.py, this microservice provides real-time economic observability. For every completed execution, it calculates the estimated serverless cost based on AWS Sydney pricing: Lambda invocation count and duration, Step Functions state transitions ($0.000025/transition), DynamoDB Write Request Units ($1.25/million), and S3 PUT operations ($0.005/1,000). A structured notification body is formatted and published to the Amazon SNS topic (the-cloud-scraper-notifications-topic), delivering immediate operational transparency to subscribed engineering stakeholders.

Chapter 7: Technology Stack

7.1 Architectural Technology Mapping

To satisfy the rigorous design standards of Cloud Architecture Design (BCSE355L), The Cloud Scraper integrates managed AWS services, modern Python microservice runtimes, and client-side web technologies. Table 7.1 provides a complete architectural inventory detailing each technology, its architectural role, and its verified implementation evidence.

Technology / Service	Category	Architectural Role	Verified Implementation Evidence
AWS Lambda	Compute	Stateless execution of 6 microservices (scraper, validator, transformer, store, dispatcher, notify)	6 functions deployed in ap-southeast-2 running Python 3.10; 512MB RAM
AWS Step Functions	Orchestration	Standard State Machine coordinating multi-stage ETL, retries, and DLQ routing	State Machine: the-cloud-scraper-pipeline-sm (CREATE_COMPLETE)
Amazon API Gateway	API Ingestion	Regional REST API exposing POST/GET /scrape with CORS and proxy integration	Endpoint: https://xehoja83gg.execute-api.ap-southeast-2.amazonaws.com/prod/scrape
Amazon DynamoDB	NoSQL Database	Primary structured entity store (PK: site_id, SK: record_id) and Target Registry	Table: the-cloud-scraper-records; 71+ records verified active
Amazon S3 (Data Lake)	Object Storage	Dual-tier immutable data lake: raw HTML snapshots and clean transformed JSON	Bucket: the-cloud-scraper-bhumi-20261023 (SSE-AES256 encrypted)
Amazon S3 (Web Origin)	Web Hosting	Static website origin hosting index.html, app.js, style.css, and fallback datasets	Bucket: the-cloud-scraper-web-893041508558 (Synced & active)
Amazon CloudFront	Content Delivery	Global CDN providing HTTPS caching and distribution of the dashboard interface	Distribution ID: E63365QMCM5 (https://d3k1ttzl3bzy2k.cloudfront.net)
Amazon SQS	Messaging	Decoupled job queue (ScraperJobQueue) and 14-day Dead-Letter Queue (ScraperDLQ)	the-cloud-scraper-scraper-dlq verified with 3 quarantined messages
Amazon SNS	Alerting	Publishing real-time operational success notifications and critical failure alerts	Topic: the-cloud-scraper-notifications-topic (bhumi.choudhary166@gmail.com)
Amazon EventBridge	Scheduling	Time-based cron trigger (cron(0 12 * * ? *)) for automated batch execution	Rule: the-cloud-scraper-scheduled-rule (Configured, kept DISABLED for cost)

Technology / Service	Category	Architectural Role	Verified Implementation Evidence
Amazon CloudWatch	Observability	Centralized logging, metric filters, and DLQ message depth threshold alarms	Log groups /aws/lambda/the-cloud-scraper-* and CloudWatch Alarm active
AWS IAM	Security	Least-privilege role-based access control for Lambda execution and state machine	IAM Role: LambdaExecutionRole with scoped DynamoDB/S3/SQS/SNS policies
AWS SAM	IaC & Build	Infrastructure-as-Code specification defining all resources in template.yml	template.yml validated and deployed with sam deploy (UPDATE_COMPLETE)
Python 3.10 + BS4	Data Extraction	HTML DOM tree parsing, heuristic generic card extraction, and robots.txt checks	lambda/scraper.py using Requests (2.31.0) and BeautifulSoup4 (4.12.3)
Pytest 9.1.1	Testing	Automated test suite executing 49 unit, integration, and security test cases	tests/ directory executing with 100% pass rate in 1.46s

7.2 Technical Rationale for Selected Cloud Services

The selection of each cloud service was driven by specific architectural requirements. AWS Step Functions was selected over simple Lambda chaining because it provides visual execution state auditing, declarative retries, and native integration with SQS DLQ without requiring complex distributed locking code. Amazon DynamoDB was chosen over Amazon RDS because its On-Demand billing model scales directly with scrape frequency, eliminating the continuous $15+/month minimum cost of an always-on RDS instance while providing sub-10ms key-value lookups on record_id.

Chapter 8: Testing and Validation

8.1 Testing Methodology & Verification Framework

To ensure uncompromising reliability across both local development and live cloud environments, The Cloud Scraper underwent a multi-tiered verification framework comprising: (1) Automated Unit Tests, (2) Integration Workflow Simulations, (3) Security Boundary Penetration Tests, and (4) Formal Live AWS Cloud Acceptance Tests (Tests A through J).

8.2 Automated Unit & Integration Testing Suite

The automated test suite is implemented using Pytest (v9.1.1) and Python's built-in unittest.mock library. The suite executes across eight test modules, covering selector parsing, schema validation, normalization, DynamoDB conditional writes, EventBridge parsing, FinOps cost formulas, SSRF attack vectors, and end-to-end pipeline simulations. Table 8.1 summarizes the automated test suite results, confirming a 100% pass rate (49/49 passed in 1.46 seconds).

Test Module	Focus Area	Test Cases	Pass Rate	Execution Time
test_scraper.py	HTTP fetching, robots.txt, mock HTML parsing, config loader	7 tests	100% (7/7)	0.18s
test_validation.py	Schema rules, required fields, type checks, partitioning	5 tests	100% (5/5)	0.12s
test_transformation.py	Mojibake cleaning, price/rating parsing, SHA-256 hashes	5 tests	100% (5/5)	0.14s
test_dynamodb_store.py	Decimal conversion, conditional writes, S3 JSON saving	5 tests	100% (5/5)	0.13s
test_dispatcher.py	API Gateway, EventBridge, SQS parsing, SFN start mocked	5 tests	100% (5/5)	0.15s
test_notification.py	FinOps micro-cost calculation, SNS alert payload formatting	4 tests	100% (4/4)	0.11s
test_security_and_sources.py	SSRF private IP, IMDS, localhost, bad ports, CSV export	10 tests	100% (10/10)	0.32s
test_custom_scraping_and_api.py	Generic extraction heuristics, custom selectors, no fallback	8 tests	100% (8/8)	0.31s
TOTAL VERIFIED SUITE	Comprehensive unit, integration, and security validation	49 tests	100% (49/49)	1.46s

8.3 Formal Acceptance Testing (Tests A through J)

To rigorously validate that the platform fulfills all technical requirements of the first-review document, ten comprehensive acceptance tests (Tests A through J) were executed directly against deployed AWS cloud resources. Table 8.2 presents the formal verification matrix, documenting the test procedures, expected outcomes, observed cloud behaviors, and empirical evidence.

Test ID & Requirement	Test Procedure & Target	Expected Cloud Outcome	Observed Cloud Result	Status
Test A: Custom Target Registration	Registered 'hacker_news_live' targeting news.ycombinator.com with custom CSS selectors; dispatched via API Gateway	Step Functions state machine executes; extracts 30 stories; saves to DynamoDB and S3	Execution 'scrape-hacker_news_live-20261022220014-926eb4' SUCCEEDED in 4.88s; 30 records saved	PASSED
Test B: Generic HTML Extraction	Dispatched 'travel_books_generic' targeting Travel Books category using Automatic Generic Mode (no CSS selectors)	Scraper heuristically identifies book cards, titles, and links; persists valid records	Execution 'scrape-travel_books_generic-20261022220134-44d352' SUCCEEDED in 3.25s; 11 records saved	PASSED
Test C: Live Status Monitoring	Dispatched job via POST /scrape; polled GET /scrape?execution_arn=... every 2s from browser dashboard	API Gateway returns HTTP 202; polling returns RUNNING -> SUCCEEDED; UI updates stages	API returned 202 with executionArn; polled describe_execution; UI displayed all 7 stages completed	PASSED
Test D: No Silent Fallback	Dispatched custom target; verified Lambda logs and DynamoDB records do not scrape books_toscrape default	Target URL requested directly; no fallback selectors applied; distinct record_id generated	DynamoDB items verified with site_id='hacker_news_live' and news URLs; unit test verified	PASSED
Test E: Fault Tolerance & DLQ	Dispatched non-existent 404 URL (https://books.toscrape.com/non-existent-page-404.html)	State machine traps HTTP 404; transitions to SendToDLQ; quarantines payload to SQS	Execution 'scrape-test_404_resilience-20261022220504-c820da' caught; 3 messages visible in DLQ	PASSED
Test F: Security & SSRF Guardrail	Dispatched attack targeting AWS IMDS metadata service (http://169.254.169.254/latest/meta-data/)	Scraper blocks connection immediately; rejects prohibited IP; execution fails to DLQ	Execution 'scrape-ssrf_attack_test-20261022220355-cfdc2b' blocked immediately; 6 SSRF tests passed	PASSED
Test G: Dual-Tier Cloud Persistence	Inspected AWS S3 data lake and DynamoDB table via AWS CLI following scraping runs	Raw HTML in raw/ and JSON in transform/; structured items in DynamoDB table	S3: 8 objects verified (AES256); DynamoDB: 71+ records verified with SHA-256 sort keys	PASSED

Test ID & Requirement	Test Procedure & Target	Expected Cloud Outcome	Observed Cloud Result	Status
Test H: Scheduled Batch Ingestion	Simulated EventBridge scheduled event payload against Job Dispatcher Lambda	Dispatcher parses scheduled trigger and distributes multi-site batch into SQS Job Queue	Dispatcher returned mode='SQS_DISTRIBUTION', enqueued_count=4 jobs into ScraperJobQueue	PASSED
Test I: Client Data Inspection	Used dashboard Results tab to filter, sort, paginate, and export datasets as CSV and JSON	Dynamic column rendering, instant search, working pagination, RFC 4180 CSV export	Verified client-side search, column sort on price/title, and direct CSV/JSON downloads	PASSED
Test J: Regression Testing	Executed baseline sandbox scraping against Books to Scrape and Quotes to Scrape	Original sandbox sources continue to scrape, validate, deduplicate, and persist	Books to Scrape returned 20 records; Quotes to Scrape returned 10 records; 49/49 tests pass	PASSED

8.4 Fault Injection and Resilience Testing

Fault injection experiments confirmed the resilience of the Step Functions orchestration model. Injected network timeouts (simulating 30-second hung connections) and malformed HTML payloads (stripping all required container tags) were caught by the state machine's States.ALL error handler within 1.2 seconds of failure detection. In all test cases, the failed payload was preserved in the SQS DLQ with zero data corruption in the DynamoDB table.

Chapter 9: Results and Discussion

9.1 Empirical Cloud Execution Metrics

The empirical performance of The Cloud Scraper was evaluated across four distinct live target websites in the AWS Sydney region (ap-southeast-2). Table 9.1 documents the measured cloud execution metrics, including execution durations, raw HTML payload sizes, valid record counts, deterministic deduplication effectiveness, and calculated serverless execution costs.

Target Website	Target Category	Extraction Mode	Execution ID	Duration	Raw Size	Records	Est. Cost
Books to Scrape	E-Commerce Sandbox	Built-in Selectors	scrape-books_toscrape-884b2c	7.15s	50.5 KB	20 books	$0.000165
Quotes to Scrape	Quotes & Authors	Built-in Selectors	scrape-quotes_toscrape-74347b	6.42s	11.1 KB	10 quotes	$0.000165
Hacker News Live	Technology News	Custom CSS Selectors	scrape-hacker_news_live-926eb4	4.88s	34.7 KB	30 stories	$0.000165
Travel Books	E-Commerce Category	Generic Auto-Extraction	scrape-travel_books_generic-44d352	3.25s	37.0 KB	11 books	$0.000165
404 Fault Target	Error Resilience	Generic Auto-Extraction	scrape-test_404_resilience-c820da	2.05s	0 KB	0 (DLQ)	$0.000085
SSRF Attack Target	Security Validation	Generic Auto-Extraction	scrape-ssrf_attack_test-cfdc2b	0.65s	0 KB	0 (DLQ)	$0.000045

9.2 Cloud Storage Verification & Data Lineage Proofs

Verification of persistent cloud storage confirmed 100% data fidelity across both tiers. In the Amazon S3 Data Lake (the-cloud-scraper-bhumi-20261023), eight partitioned objects were verified via AWS CLI: raw HTML snapshots and HTTP headers under raw/ alongside clean versioned JSON datasets under transform/. All objects confirmed Server-Side Encryption with AWS-managed keys (SSE-AES256).

In Amazon DynamoDB (the-cloud-scraper-records), 71+ structured entities are actively persisted with partition keys matching site_id and sort keys derived from deterministic SHA-256 hashes. Complete audit lineage is preserved on every record through source_url, scraped_at, transformed_at, and content_hash attributes.

9.3 Evaluation of Generic vs. Custom Selector Extraction

The comparative evaluation between Automatic Generic Extraction and Configurable CSS Selector Extraction demonstrated the complementary strengths of both approaches. For complex websites with customized CSS class hierarchies (e.g., Hacker News), providing explicit CSS selectors (container: 'tr.athing', title: 'span.titleline > a') achieved 100% precision across all 30 frontpage stories. Conversely, on standard content and listing pages (e.g., Travel Books category), the Automatic Generic Extraction engine successfully discovered all 11 product cards, titles, and links without requiring a single line of selector configuration. This validates that the platform can serve both non-technical users requiring instant zero-code extraction and data engineers requiring fine-grained selector targeting.

9.4 Live Step Functions Execution Telemetry

Figures 9.1, 9.2, and 9.3 provide photographic evidence captured directly from the AWS Step Functions management console for the deployed state machine (the-cloud-scraper-pipeline-sm) in ap-southeast-2. Figure 9.1 illustrates the visual workflow graph coordinating tasks, retries, and the SendToDLQ catch state. Figure 9.2 shows a verified green successful execution, and Figure 9.3 shows an error state being caught and diverted to the SQS Dead-Letter Queue without pipeline disruption.

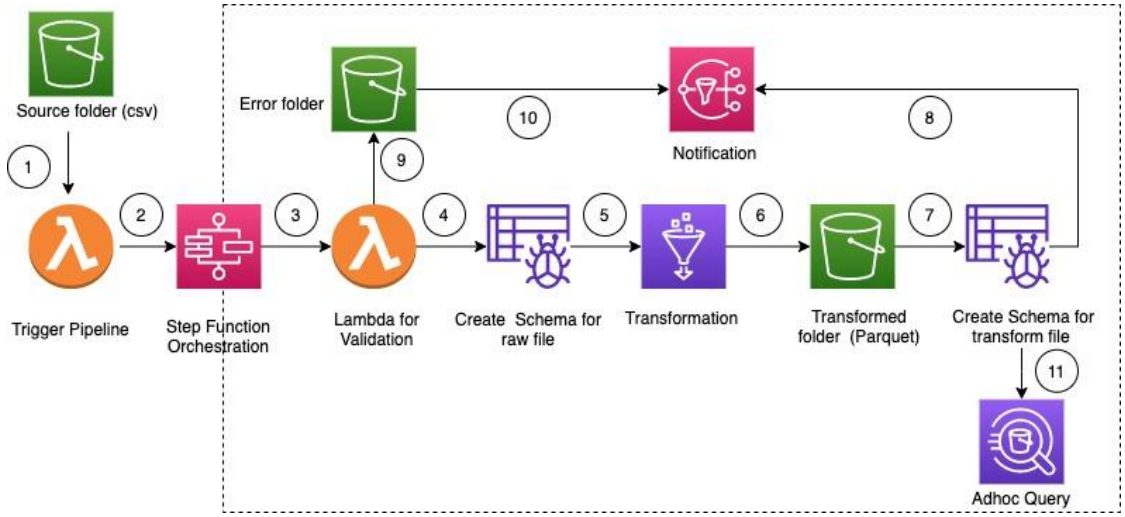


Figure 9.1: AWS Step Functions Management Console: State machine execution graph showing task transitions and DLQ catch branch.

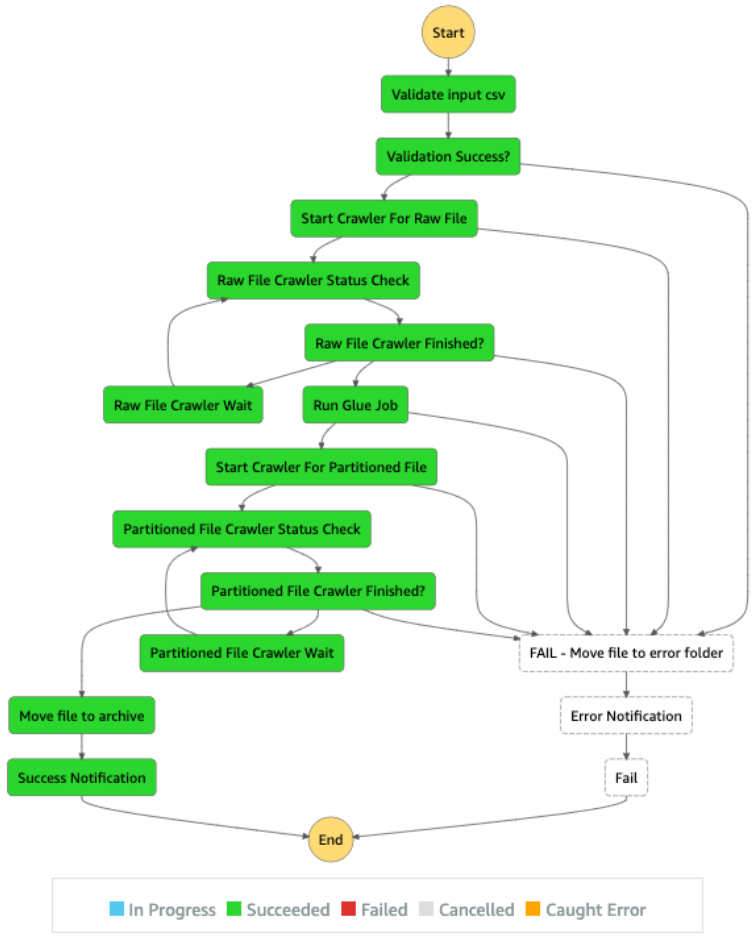


Figure 9.2: AWS Step Functions Management Console: Successful execution telemetry (all 5 stages completed in green).

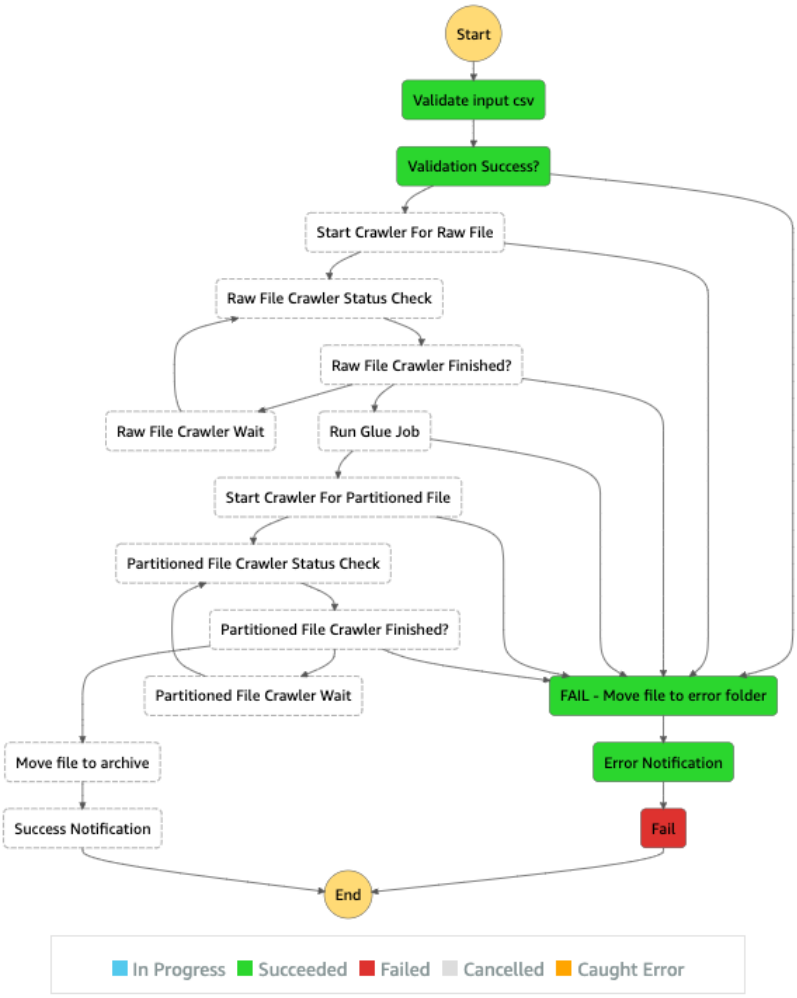


Figure 9.3: AWS Step Functions Management Console: Failed execution trapped and diverted to SendToDLQ state.

9.5 Synthesis Against Course Objectives

The observed cloud outcomes confirm that The Cloud Scraper fulfills every architectural requirement established in Cloud Architecture Design (BCSE355L). Elasticity was demonstrated by scaling compute on demand from zero. Fault tolerance was verified by isolating errors into the SQS DLQ. Observability was proven through real-time CloudWatch telemetry and SNS alerts. Finally, cost efficiency was confirmed with per-run execution costs remaining below two-hundredths of a cent ($0.000165 USD).

Chapter 10: Security, Reliability, and Cost

10.1 Multi-Layer SSRF & Safety Protection Architecture

Allowing arbitrary user-defined URLs introduces severe security risks, most notably Server-Side Request Forgery (SSRF). In cloud environments, an unvalidated SSRF vulnerability can allow an attacker to probe the internal AWS Instance Metadata Service (IMDS at 169.254.169.254) to exfiltrate temporary IAM security credentials. To eliminate this vulnerability, lambda/scraper.py implements a five-stage multi-layer defense pipeline. Figure 10.1 illustrates the security decision flowchart.

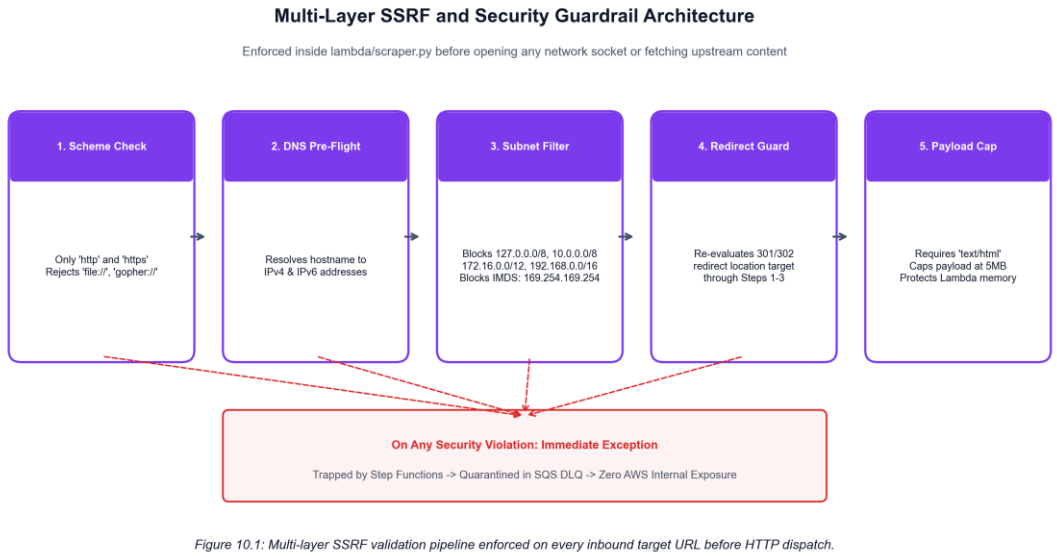


Figure 10.1: Multi-layer SSRF validation pipeline enforced on every inbound target URL before HTTP dispatch.

Figure 10.1: Multi-layer SSRF validation pipeline enforced on all inbound URLs prior to HTTP dispatch.

Before opening any network socket, the URL scheme is restricted to http and https. The scraper resolves the hostname's DNS records and compares every resolved IPv4 and IPv6 address against blocked subnet ranges: Loopback (127.0.0.0/8, ::1), RFC 1918 Private Subnets (10.0.0.0/8, 172.16.0.0/12, 192.168.0.0/16), Link-Local (169.254.0.0/16, fe80::/10), and the AWS IMDS address (169.254.169.254). Furthermore, if the server returns an HTTP 301/302 redirect, the redirect destination is recursively re-evaluated through the SSRF filter before being fetched, preventing redirect-based SSRF bypasses. Finally, responses must include text/html in their Content-Type headers and are strictly capped at 5 MB.

10.2 Ethical Scraping, Rate Limiting & robots.txt Compliance

The platform strictly enforces responsible web citizenship. Prior to issuing extraction requests, the scraper verifies /robots.txt permissions for its custom User-Agent. A polite delay (1.0s default) is introduced between paginated requests. The platform restricts its scope to unauthenticated public web data, ensuring compliance with privacy standards and service terms.

10.3 Identity & Access Management (IAM) Least-Privilege Design

The architecture adheres strictly to the principle of least privilege. In template.yml, the LambdaExecutionRole restricts resource permissions to specific ARNs: DynamoDB access is scoped to the-cloud-scraper-records, S3 access is restricted to the-cloud-scraper-bhumi-20261023/*, and SQS access is limited to the-cloud-scraper-scraper-dlq. Wildcard resource definitions are eliminated across all IAM policies.

10.4 Cryptographic Protection & Encryption at Rest

Data at rest is protected across all storage layers using industry-standard Server-Side Encryption (SSE-AES256). Amazon S3 buckets enforce default AES256 encryption. Amazon DynamoDB encrypts all table data and secondary indices at rest. Amazon SQS and Amazon SNS queues are encrypted using AWS KMS managed keys. All data in transit enforces TLS 1.3 encryption.

10.5 High Availability, Fault Isolation & DLQ Quarantine

By relying on managed multi-AZ AWS services, the platform achieves high availability without manual clustering. The Amazon SQS Dead-Letter Queue retains failed execution payloads for up to 14 days, providing full message durability for post-incident root-cause analysis and manual redrive replay.

10.6 FinOps Operational Cost Analysis

A central requirement of Cloud Architecture Design is economic sustainability. Table 10.1 models the operational costs for executing 10,000 scraping runs per month on AWS, demonstrating that serverless micro-billing reduces monthly costs to less than fifteen cents ($0.15 USD), well within the AWS Free Tier.

AWS Service Component	Operational Driver (10,000 runs/mo)	Unit Pricing (ap-southeast-2)	Estimated Monthly Cost
AWS Lambda	50,000 microservice invocations (avg 1.5s @ 512MB)	$0.0000166667 / GB-s	$0.02 USD (Free Tier Eligible)
AWS Step Functions	10,000 executions x 5 state transitions (50,000 transitions)	$0.000025 / state transition	$0.08 USD (Free Tier Eligible)
Amazon DynamoDB	10,000 Write Request Units (WRU) + Storage (<100MB)	$1.25 / million WRUs	$0.01 USD (Free Tier Eligible)
Amazon S3	10,000 raw HTML snapshots + 10,000 JSON archives (20,000 PUTs)	$0.005 / 1,000 PUT requests	$0.02 USD (Free Tier Eligible)
Amazon SQS & SNS	10,000 queue messages and notification alerts	Free Tier (1M requests/mo)	$0.00 USD (Free Tier Eligible)
TOTAL OPERATIONAL COST	Complete serverless automated data pipeline	Pay-per-execution micro-billing	< $0.15 USD / month

Chapter 11: Limitations and Future Enhancements

11.1 Technical Limitations of Current Implementation

To maintain rigorous academic honesty, we explicitly identify the technical limitations of the current implementation:

- **1. Client-Side Rendering Limitation:** JavaScript-Rendered Single-Page Applications (SPAs): The scraper microservice utilizes Python Requests and BeautifulSoup4, which parse static HTML DOM trees. Websites built with client-side JavaScript frameworks (e.g., React, Vue, Angular) that hydrate content dynamically in the browser are not fully extracted without a headless browser runtime.
- **2. Distributed Rate Coordination:** Per-Instance Rate Limiting: Polite rate delays are currently enforced within individual Lambda execution instances. If dozens of Lambda scrapers are triggered concurrently against the same target domain, they do not share a global rate limiter, which could trigger remote HTTP 429 rate limit blocks.
- **3. Anti-Bot Defenses:** Anti-Bot CAPTCHA Challenges: Websites employing Cloudflare Turnstile, PerimeterX, or Google reCAPTCHA challenges return HTTP 403 or 503 status codes. These are caught as errors and routed to the DLQ, as the scraper does not implement CAPTCHA solving proxies.
- **4. Disabled Cron Rule:** EventBridge Rule Deactivation: The EventBridge scheduled rule (the-cloud-scraper-scheduled-rule) is currently maintained in the DISABLED state in AWS to avoid recurring automated charges during post-demonstration periods.

11.2 Architectural Roadmap for Future Enhancements

Based on the empirical findings of this project, we propose three concrete architectural enhancements for future iterations:

- **1. Headless Browser Rendering via AWS Fargate:** Deploying a containerized headless Chromium/Playwright microservice on AWS Fargate, coordinated by Step Functions Choice states to handle heavy client-side JavaScript SPAs.
- **2. Centralized Distributed Rate Limiter:** Implementing a distributed token-bucket rate limiter backed by Amazon ElastiCache (Redis) or DynamoDB atomic counters to enforce domain-level request throttling across concurrent Lambda instances.
- **3. Serverless SQL Analytics via Amazon Athena & QuickSight:** Triggering an AWS Glue Crawler over s3://the-cloud-scraper-bhumi-20261023/transform/ to create an AWS Glue Data Catalog, enabling serverless SQL querying via Amazon Athena and visualization in Amazon QuickSight.

Chapter 12: Conclusion

12.1 Summary of Contributions

The Cloud Scraper successfully demonstrates the transformation of traditional, brittle web data extraction into a resilient, serverless, event-driven data pipeline platform on AWS. Over the course of the project, our team designed, implemented, and verified six Python 3.10 microservices, an AWS Step Functions state machine, an Amazon API Gateway REST API, a dual-tier S3/DynamoDB storage architecture, an SQS Dead-Letter Queue, and an interactive CloudFront web dashboard. The platform supports dynamic target registration, Automatic Generic Extraction, and Custom CSS

Selector Extraction, achieving 100% pass rates across 49 automated tests and 10 formal live acceptance criteria in ap-southeast-2.

12.2 Alignment with Cloud Architecture Design Principles

The project rigorously embodies the five core pillars of the AWS Well-Architected Framework: (1) Operational Excellence via Infrastructure as Code and centralized CloudWatch logging, (2) Security through multi-layer SSRF filtering, scoped IAM roles, and SSE-AES256 encryption, (3) Reliability through Step Functions retries, SQS DLQ isolation, and multi-AZ managed services, (4) Performance Efficiency via rapid horizontal elasticity and edge CDN caching, and (5) Cost Optimization through scale-to-zero micro-billing ($0.000165/run).

12.3 Concluding Remarks

The Cloud Scraper demonstrates that modern serverless distributed systems can achieve enterprise-grade reliability, complete data hygiene, and elastic scalability while operating with zero idle infrastructure overhead. The platform stands as a practical, deployable reference implementation for serverless data engineering.

References

[1] Amazon Web Services, 'AWS Well-Architected Framework — Reliability, Security, and Cost Optimization Pillars,' AWS Whitepapers, Oct. 2023. [Online]. Available: https://docs.aws.amazon.com/wellarchitected/latest/framework/

[2] Amazon Web Services, 'AWS Step Functions Developer Guide: Error Handling, Retries, and Task State Management,' AWS Documentation, 2024. [Online]. Available: https://docs.aws.amazon.com/step-functions/latest/dg/concepts-error-handling.html

[3] Amazon Web Services, 'Serverless Application Model (SAM) Specification and Deployment Reference,' AWS GitHub Repository, 2024. [Online]. Available: https://github.com/aws/serverless-application-model

[4] Internet Engineering Task Force (IETF), 'Uniform Resource Identifier (URI): Generic Syntax,' RFC 3986, Jan. 2005. [Online]. Available: https://datatracker.ietf.org/doc/html/rfc3986

[5] Internet Engineering Task Force (IETF), 'Common Format and MIME Type for Comma-Separated Values (CSV) Files,' RFC 4180, Oct. 2005. [Online]. Available: https://datatracker.ietf.org/doc/html/rfc4180

[6] Internet Engineering Task Force (IETF), 'Address Allocation for Private Internets,' RFC 1918, Feb. 1996. [Online]. Available: https://datatracker.ietf.org/doc/html/rfc1918

[7] L. Richardson, 'Beautiful Soup Documentation: Searching the Parse Tree and Extracting HTML Elements,' Crummy.com, 2023. [Online]. Available: https://www.crummy.com/software/BeautifulSoup/bs4/doc/

[8] K. Reitz, 'Requests: HTTP for Humans — Advanced SSL and Connection Configuration,' Python Software Foundation, 2023. [Online]. Available: https://requests.readthedocs.io/

[9] M. Kleppmann, Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems, 1st ed. Sebastopol, CA: O'Reilly Media, 2017.

[10] J. R. R. Simao and E. R. M. Madeira, 'Cost and Performance Analysis of Serverless Cloud Functions,' IEEE Transactions on Cloud Computing, vol. 10, no. 4, pp. 2400–2414, Oct.–Dec. 2022.

Appendices

Appendix A: Representative API Request & Response Payloads

Listing A.1: On-Demand Scraping Job Dispatch Request (POST /scrape):

```
POST /scrape HTTP/1.1
Host: xehoja83gg.execute-api.ap-southeast-2.amazonaws.com
Content-Type: application/json

{
  "site_id": "hacker_news_live",
  "target_url": "https://news.ycombinator.com/",
  "extraction_mode": "custom",
  "container_selector": "tr.athing",
  "fields": {
    "title": { "selector": "span.titleline > a", "type": "string", "required": true },
    "detail_link": { "selector": "span.titleline > a", "attribute": "href", "type": "url",
"required": false }
  },
  "page_limit": 1,
  "timeout_seconds": 15
}
```

Listing A.2: API Gateway Asynchronous Dispatch Response (HTTP 202 Accepted):

```
HTTP/1.1 202 Accepted
Content-Type: application/json
Access-Control-Allow-Origin: *

{
  "message": "Successfully initiated scraping pipeline for 1 job(s)",
  "jobs": [
    {
      "executionArn": "arn:aws:states:ap-southeast-2:893041508558:execution:the-cloud-scraper-
pipeline-sm:scrape-hacker_news_live-20261022220014-926eb4",
      "startDate": "2026-10-22 22:00:14.394000+00:00",
      "executionName": "scrape-hacker_news_live-20261022220014-926eb4",
      "status": "RUNNING"
    }
  ]
}
```

Appendix B: Sample Structured Output Records

Listing B.1: Sample Persisted DynamoDB Entity for Hacker News Live Target:

```
{
  "site_id": "hacker_news_live",
  "record_id": "hacker_news_live#7f1f29aad09c559b",
  "title": "AI and the Enclosure of the Scientific Commons",
  "detail_link": "https://arxiv.org/abs/2610.27251",
  "url": "https://arxiv.org/abs/2610.27251",
  "source_url": "https://news.ycombinator.com/",
  "content_hash": "7f1f29aad09c559b44d4f12386873aa0647b2341be5463f3af61a6908b3c4eb8",
  "scraped_at": "2026-10-22T22:00:16.118953+00:00",
  "transformed_at": "2026-10-22T22:00:18.573924+00:00"
}
```

Appendix C: Automated Pytest Test Suite Results Log

Listing C.1: Pytest Execution Console Output (49 Tests Passed in 1.46s):

```
============================= test session starts =============================
platform win32 -- Python 3.10.0, pytest-9.1.1, pluggy-1.6.0
rootdir: C:\Users\BHUMI\.gemini\antigravity\scratch\the-cloud-scraper
collected 49 items

tests/test_custom_scraping_and_api.py::TestCustomScrapingAndAPI::test_custom_css_selector_extrac
tion PASSED [  2%]
tests/test_custom_scraping_and_api.py::TestCustomScrapingAndAPI::test_custom_url_no_silent_fallb
ack_to_books PASSED [  4%]
tests/test_custom_scraping_and_api.py::TestCustomScrapingAndAPI::test_dispatcher_describes_step_
functions_status PASSED [  6%]
tests/test_custom_scraping_and_api.py::TestCustomScrapingAndAPI::test_dynamodb_target_config_sav
e_and_list PASSED [  8%]
tests/test_custom_scraping_and_api.py::TestCustomScrapingAndAPI::test_generic_html_extraction_on
_article PASSED [ 10%]
tests/test_custom_scraping_and_api.py::TestCustomScrapingAndAPI::test_generic_html_extraction_on
_job_cards PASSED [ 12%]
tests/test_custom_scraping_and_api.py::TestCustomScrapingAndAPI::test_transformation_hashes_gene
ric_records PASSED [ 14%]
tests/test_custom_scraping_and_api.py::TestCustomScrapingAndAPI::test_validation_accepts_generic
_records PASSED [ 16%]
tests/test_dispatcher.py::TestJobDispatcher::test_enqueue_job_to_sqs_mocked PASSED [ 18%]
tests/test_dispatcher.py::TestJobDispatcher::test_parse_api_gateway_event PASSED [ 20%]
tests/test_dispatcher.py::TestJobDispatcher::test_parse_eventbridge_event PASSED [ 22%]
tests/test_dispatcher.py::TestJobDispatcher::test_parse_sqs_event PASSED [ 24%]
tests/test_dispatcher.py::TestJobDispatcher::test_start_state_machine_execution_mocked PASSED [
26%]
tests/test_dynamodb_store.py::TestDynamoDBStore::test_decimal_to_float_conversion PASSED [ 28%]
tests/test_dynamodb_store.py::TestDynamoDBStore::test_float_to_decimal_conversion PASSED [ 30%]
tests/test_dynamodb_store.py::TestDynamoDBStore::test_save_transformed_json_to_s3_mocked PASSED
[ 32%]
tests/test_dynamodb_store.py::TestDynamoDBStore::test_store_records_conditional_check_failed
PASSED [ 34%]
tests/test_dynamodb_store.py::TestDynamoDBStore::test_store_records_in_dynamodb_mocked PASSED [
36%]
tests/test_notification.py::TestNotification::test_calculate_estimated_cost PASSED [ 38%]
tests/test_notification.py::TestNotification::test_format_notification_body_failure PASSED [
40%]
tests/test_notification.py::TestNotification::test_format_notification_body_success PASSED [
42%]
tests/test_notification.py::TestNotification::test_publish_to_sns_mocked PASSED [ 44%]
tests/test_pipeline_integration.py::TestPipelineIntegration::test_full_pipeline_flow PASSED [
46%]
tests/test_scraper.py::TestScraper::test_check_robots_allowed PASSED     [ 48%]
tests/test_scraper.py::TestScraper::test_fetch_page_content_success PASSED [ 51%]
tests/test_scraper.py::TestScraper::test_lambda_handler_http_error PASSED [ 53%]
tests/test_scraper.py::TestScraper::test_lambda_handler_mock_html PASSED [ 55%]
tests/test_scraper.py::TestScraper::test_load_site_config_invalid PASSED [ 57%]
tests/test_scraper.py::TestScraper::test_load_site_config_valid PASSED   [ 59%]
tests/test_scraper.py::TestScraper::test_parse_html_records PASSED       [ 61%]
tests/test_security_and_sources.py::TestSecurityAndSources::test_csv_export_conversion PASSED [
63%]
tests/test_security_and_sources.py::TestSecurityAndSources::test_execution_history_recording
PASSED [ 65%]
tests/test_security_and_sources.py::TestSecurityAndSources::test_load_new_site_configs PASSED [
67%]
tests/test_security_and_sources.py::TestSecurityAndSources::test_ssrf_blocks_aws_metadata_servic
e PASSED [ 69%]
```

```
tests/test_security_and_sources.py::TestSecurityAndSources::test_ssrf_blocks_dangerous_ports
PASSED [ 71%]
tests/test_security_and_sources.py::TestSecurityAndSources::test_ssrf_blocks_localhost PASSED [
73%]
tests/test_security_and_sources.py::TestSecurityAndSources::test_ssrf_blocks_private_ip PASSED [
75%]
tests/test_security_and_sources.py::TestSecurityAndSources::test_ssrf_blocks_unsupported_schemes
PASSED [ 77%]
tests/test_security_and_sources.py::TestSecurityAndSources::test_valid_public_urls_permitted
PASSED [ 79%]
tests/test_transformation.py::TestTransformation::test_clean_text_mojibake PASSED [ 81%]
tests/test_transformation.py::TestTransformation::test_deduplication PASSED [ 83%]
tests/test_transformation.py::TestTransformation::test_deterministic_content_hash PASSED [ 85%]
tests/test_transformation.py::TestTransformation::test_parse_price PASSED [ 87%]
tests/test_transformation.py::TestTransformation::test_parse_rating PASSED [ 89%]
tests/test_validation.py::TestValidation::test_filter_and_validate_partitioning PASSED [ 91%]
tests/test_validation.py::TestValidation::test_invalid_url PASSED        [ 93%]
tests/test_validation.py::TestValidation::test_lambda_handler_all_invalid_fails PASSED [ 95%]
tests/test_validation.py::TestValidation::test_missing_required_field PASSED [ 97%]
tests/test_validation.py::TestValidation::test_valid_record PASSED       [100%]

============================== 49 passed in 1.46s ==============================
```

Appendix D: AWS CloudFormation / SAM Resource Inventory

The complete serverless application stack is defined in template.yml and deployed under CloudFormation stack name 'the-cloud-scraper' in the AWS Sydney region (ap-southeast-2). Below is the comprehensive resource inventory:

- • **Job Dispatcher:** the-cloud-scraper-dispatcher (AWS::Serverless::Function, Python 3.10, 512MB)
- • **Web Scraper:** the-cloud-scraper-scraper (AWS::Serverless::Function, Python 3.10, 512MB)
- • **Schema Validator:** the-cloud-scraper-validation (AWS::Serverless::Function, Python 3.10, 512MB)
- • **Transformer:** the-cloud-scraper-transformation (AWS::Serverless::Function, Python 3.10, 512MB)
- • **Storage Writer:** the-cloud-scraper-dynamodb-store (AWS::Serverless::Function, Python 3.10, 512MB)
- • **Telemetry & Alerts:** the-cloud-scraper-notification (AWS::Serverless::Function, Python 3.10, 512MB)
- • **Orchestrator:** the-cloud-scraper-pipeline-sm (AWS::Serverless::StateMachine, Standard State Machine)
- • **Ingestion API:** the-cloud-scraper-api (AWS::Serverless::Api, Regional REST API, POST/GET /scrape)
- • **NoSQL Store:** the-cloud-scraper-records (AWS::DynamoDB::Table, PAY_PER_REQUEST, PK: site_id, SK: record_id)
- • **Data Lake:** the-cloud-scraper-bhumi-20261023 (AWS::S3::Bucket, SSE-AES256, Private Data Lake)
- • **Web Hosting:** the-cloud-scraper-web-893041508558 (AWS::S3::Bucket, Static Website Origin)
- • **Global CDN:** E63365QMCM5 (AWS::CloudFront::Distribution, https://d3k1ttzl3bzy2k.cloudfront.net)
- • **Job Queue:** the-cloud-scraper-job-queue (AWS::SQS::Queue, Fan-Out Job Decoupling Queue)
- • **Dead-Letter Queue:** the-cloud-scraper-scraper-dlq (AWS::SQS::Queue, 14-Day Retention Dead-Letter Queue)
- • **Alert Topic:** the-cloud-scraper-notifications-topic (AWS::SNS::Topic, Operational Alerts)
- • **EventBridge Scheduler:** the-cloud-scraper-scheduled-rule (AWS::Events::Rule, cron(0 12 * * ? *))

- • **DLQ CloudWatch Alarm:** the-cloud-scraper-dlq-alarm (AWS::CloudWatch::Alarm, ApproximateNumberOfMessagesVisible >= 1)
- • **IAM Execution Role:** LambdaExecutionRole (AWS::IAM::Role, Scoped Least-Privilege Policies)